

LOGPRESS PROJECT · WEEK 6

# LogPress 주간 발표

RCA 증거 보존형 KV Cache 압축 — 실험 · 설계 · 평가

PART A

**압축 실험**

Thunderbird · SnapKV / OA 보존율

PART C

**Phase 1 실험설계**

Query-free Cross-Line Attention

PART D

**평가 설계**

Ablation · K 지선다 RCA

2026. 08. 31

---

PART A

# Thunderbird 압축 실험

1 시간 window 로그를 KV 압축했을 때 alert line 이 실제로 얼마나 살아남는지 —  
SnapKV · ObservedAttention baseline 을 재현하고 line-level 보존율을 측정한다 .

# Thunderbird 로그 기반 KV Cache 압축 Alert 보존 실험

SnapKV · ObservedAttention baseline 재현 및 line-level retention 비교

**6,022**

선택한 1시간 window의 로그 수  
window\_id = 314927

**121**

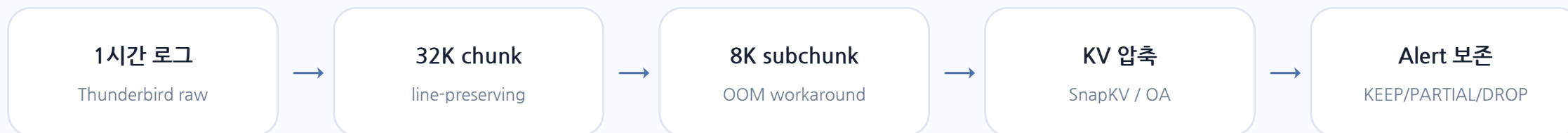
해당 window의 alert 로그 수  
label은 모델 입력에서 제거

**2 baselines**

SnapKV + ObservedAttention  
실제 생존 token 위치 추적

## 이번 주 핵심

32K 입력에서 메모리 한계를 확인한 뒤, line을 자르지 않는 약 8K subchunk 방식으로 전환하여 alert line이 압축 후 실제로 얼마나 살아남는지까지 측정했다.



# 1시간 데이터를 line 단위로 안전하게 분할

**475,068**

label 제거 후 총 token

**15**

약 32K chunk 개수

## 전처리 원칙

- raw 첫 필드의 alert label은 metadata에만 보존하고 모델 입력에서는 제거 → label leakage 방지
- 다음 line이 token limit을 넘으면 새 chunk 시작 → 한 로그 line을 중간에서 자르지 않음
- token offset을 char span과 연결해 token → 원본 line 역매핑 가능

집중 평가 대상: chunk13

**289**

logs

**32,727**

tokens

**54**

alerts

| Subchunk | Logs | Tokens | Alerts |
|----------|------|--------|--------|
| sub0     | 65   | 8,123  | 17     |
| sub1     | 74   | 8,135  | 11     |
| sub2     | 73   | 8,153  | 14     |
| sub3     | 75   | 8,174  | 12     |
| sub4     | 2    | 142    | 0      |

## 왜 8K로 다시 나눴나?

현재 환경에서 32K prefill이 attention 계산 중 OOM 발생. 데이터 정의는 32K chunk로 유지하고, baseline 검증만 8K subchunk에서 독립 실행했다.

BASELINE RESULTS

# Alert 54개 기준 SnapKV vs ObservedAttention

| 압축률 | 방법                | KEEP | PARTIAL | DROP | Strict | Any-token |
|-----|-------------------|------|---------|------|--------|-----------|
| 0.5 | SnapKV            | 19   | 34      | 1    | 35.2%  | 98.1%     |
| 0.5 | ObservedAttention | 14   | 40      | 0    | 25.9%  | 100.0%    |
| 0.7 | SnapKV            | 8    | 41      | 5    | 14.8%  | 90.7%     |
| 0.7 | ObservedAttention | 8    | 46      | 0    | 14.8%  | 100.0%    |
| 0.9 | SnapKV            | 2    | 31      | 21   | 3.7%   | 61.1%     |
| 0.9 | ObservedAttention | 4    | 17      | 33   | 7.4%   | 38.9%     |



Strict = alert line의 모든 token 생존 · Any-token = alert line에서 1개 이상 token 생존

관찰

ObservedAttention은 압축률 0.5~0.7에서 54개 alert를 하나도 완전히 DROP하지 않았지만, 0.9에서는 DROP이 33개로 급증했다. SnapKV는 0.5에서 완전 보존(KEEP)이 더 많고, 0.9에서는 최소 생존도 더 안정적이었다.

# 현재 결과의 해석 범위와 다음 실험

## 중요한 제한점

이번 결과는 32K context를 한 번에 압축한 결과가 아니다.



- subchunk 밖 context가 score 계산에 포함되지 않음
- 압축 budget과 attention context가 각 subchunk에서 다시 시작
- 따라서 현재 수치는 8K 기반 preliminary baseline으로만 해석

## NEXT

### 1 32K 재실험

서버 GPU 확보 후 chunk13 전체 context에서 재검증

### 2 Coverage 추가

alert별 kept/total token 비율과 50%·90% threshold

### 3 Baseline 확장

H2O 등 추가 baseline과 동일 조건 비교

### 4 Long context

가능하면 64K/128K까지 단계적으로 확장

## 이번 주 산출물

SNAPKV\_ALERT\_RETENTION\_RESULTS.md · OBSERVED\_ATTENTION\_ALERT\_RETENTION\_RESULTS.md  
chunk13\_subchunk.py + tracking baseline code · Git push 완료

## 다음 목표

서버 GPU에서 32K chunk를 통째로 돌려 subchunk 경계 효과를 제거한 baseline 결과를 확보하고, 이후 alert token coverage까지 평가 지표를 정교화한다.

---

PART C

# Phase 1 실험 설계

질문 없이 로그만 넣었을 때 Alert 와 Normal 의 line-to-line attention 구조가 반복적으로 달라지는가 —  
압축기의 핵심 원리 (Cross-Line Attention) 를 검증하는 1 차 실험 설계 .

## PHASE 1 EXPERIMENT

# LogPress Phase 1 실험 설계

Thunderbird\_2k 기반 소표본 Gate 실험  
Query-free Cross-Line Attention

## 핵심 질문

“ 질문 없이 로그만 넣었을 때 ,  
Alert 와 Normal 의 line-to-line  
attention 구조가  
반복적으로 달라지는가 ?”

→ RCA 정확도나 KV 압축 성능을 직접 증명하는 단계가 아니라  
line-level 관계 신호의 존재 여부를 확인하는 1 차 검증

# 1. 실험이 묻는 질문

Query-free = 사용자 질문 /Task instruction 없이 Logs only 로 forward

## ① Logs only

[LINE 0] ...

[LINE 1] ...

...

[LINE 20] ...

Question: 없음

## ② Qwen forward

Transformer 내부의  
Self-Attention(Q/K/V) 은 그대로 존재

사용자 질문이 없기 때문에  
질문 토큰이 attention 을 유도하지 않음

## ③ Line 관계 관찰

Line 11~20 → Line 10  
Cross-Line Attention

Alert/Normal 에 따라  
관계 score 분포가 달라지는지 확인

### 주의

Query-free 는 “Transformer 의 Query vector(Q) 가 없다” 는 뜻이 아니다 . 사용자 질문이 없다는 뜻이다 .

성공 시 주장 : “Alert/Normal 상태와 연관된 line-level attention 신호가 존재함을 시사” · 아직 root cause 검출 주장은 하지 않음

## 2. 표본 단위 : 21 줄 Window

앞 10 줄 + Target 1 줄 + 뒤 10 줄 , Target 은 항상 Line 10



이전 문맥

Target

이후 문맥 : Target 참조 관찰

### Alert Window

- Line 10 만 Alert
- Window 내부 추가 Alert 제외
  - 같은 incident/burst 는 대표 1 개
- Window 간 raw line overlap 최소화

### Normal Control

- Line 10 은 Normal
- 21 줄 전체 Alert 없음
  - 동일한 Line 10 위치 유지
- token 구조가 비슷한 후보와 1:1 매칭

### 표본 수

목표 : 30 Alert + 30 Normal  
= 30 pair / 60 Window

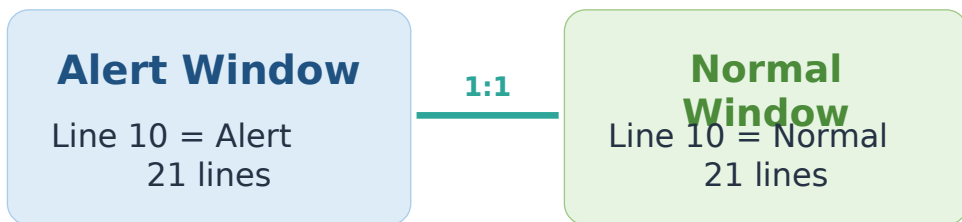
20 pair 미만이면 GO/NO-GO 확정보다  
Pilot/ 탐색 결과로 해석

### 3. Matching 과 Layer / Head 별 Token 처리

Matching 은 입력 표본 단계에서 한 번만 수행하고 , 같은 token 위치를 모든 Layer/Head 에서 공통으로 사용

#### Alert ↔ Normal 1:1 Matching

목적 : 길이와 실제 token 위치 차이를 줄여  
Alert / Normal 상태 차이를 더 공정하게 비교



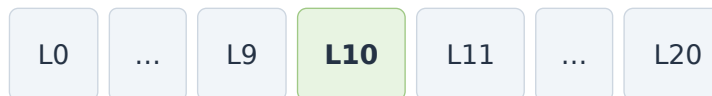
#### Matching 에 사용하는 값

|                      |                                    |
|----------------------|------------------------------------|
| <b>target_tokens</b> | Line 10 자체의 token 수                |
| <b>prefix_tokens</b> | Line 0~9 token 수 → 실제 Target 위치 통제 |
| <b>total_tokens</b>  | 21 줄 전체 context 길이 통제              |

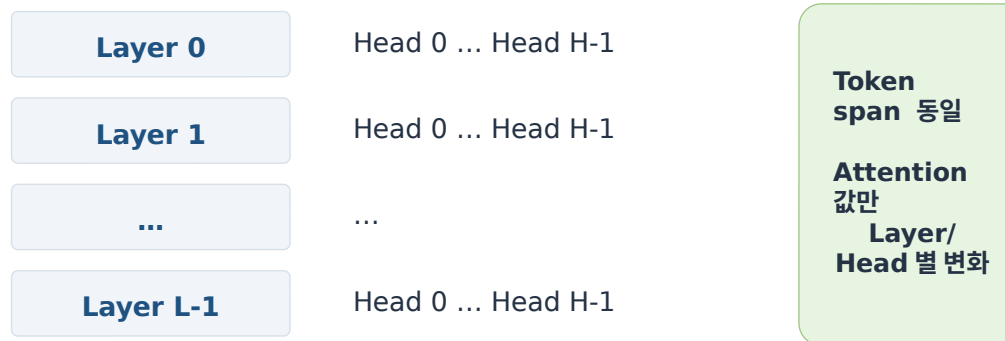
#### Layer / Head 별 Token 은 어떻게 처리 ?

Tokenization 은 Window 당 1 번만 수행

같은 token sequence



↓ 같은 token index / line span 을 모든 Layer/Head 에 재사용

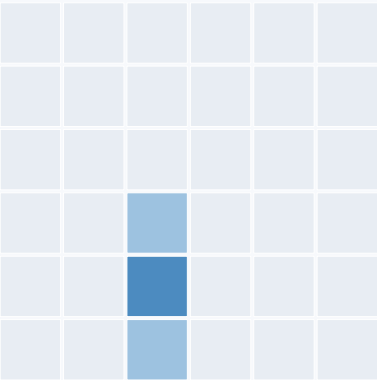


같은 token span → 각 (layer, head) 에서  $R_{ij}$  계산 →  $S_{10}^{(l,h)}$  저장 → Head 평균 / Layer 평균

# 4. Token Attention → Line Score

토큰 단위 attention 을 line-to-line 관계 점수로 집계

## Token-to-token Attention



source query tokens

target  
line  
tokens

### $R_{ij}$ (Primary)

query line  $i$  의 토큰별 attention 을 평균내고  
target line  $j$  전체가 받은 attention mass 는  
보존

Same-line  $R_{ii}$  는 제외 , 재정규화는 하지 않음

### $R_{ij}$ normalized (Secondary)

target token 수로도 나눠  
긴 Target 자체가 score 를 키우는지 확인

### Target score $S_{10}$

$$S_{10} = \text{mean}( R_{11,10} \dots R_{20,10} )$$

Line 11~20 이  
평균적으로 Line 10 을  
얼마나 참조하는가

Window 당 1 개 Primary score

## 5. AUC 평가 및 판정 기준

$S_{10}$ 의 절대값보다 Alert와 Normal의 순위 분리를 보고, Phase 1에서는 AUC 하나로 1차 Gate를 판정

### AUC의 의미

Alert 1개와 Normal 1개를 임의로 뽑았을 때  $S_{10}$  점수가 어느 집단 쪽으로 더 자주 정렬되는지 보는 순위 기반 지표

- $AUC = 0.50 \rightarrow$  분리 없음
- $AUC = 0.70 \rightarrow$  Alert 쪽으로 강한 분리
- $AUC = 0.30 \rightarrow$  반대 방향의 강한 분리

핵심 : 0.5에서 멀수록 분리 신호가 강함

### 예시 : Alert가 대체로 더 높은 경우



### 효과 기준

$|AUC - 0.5| \geq 0.15$   
 $\rightarrow AUC \geq 0.65$  또는  $AUC \leq 0.35$   
방향은 높음 / 낮음 어느 쪽이든 허용

### AUC만으로 1차 판정

#### GO

- $|AUC - 0.5| \geq 0.15$
- $AUC \geq 0.65$  또는  $\leq 0.35$

#### BOUNDARY

- $0.05 \leq |AUC - 0.5| < 0.15$
- $0.55 \sim 0.65$  또는  $0.35 \sim 0.45$

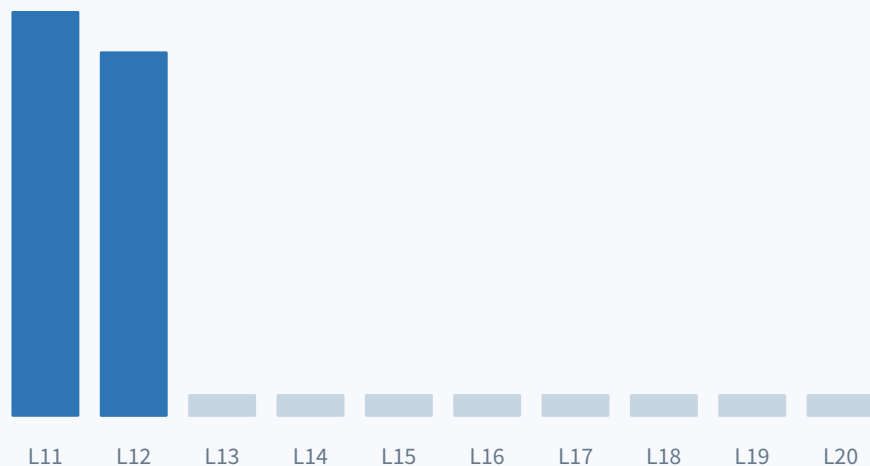
#### NO-GO

- $|AUC - 0.5| < 0.05$
- AUC가  $0.45 \sim 0.55$  부근

## 6. 예상 결과와 가장 큰 위험 : 신호 희석

Primary 기준은 유지하고 Secondary/Exploratory 분석으로 “왜 약한지” 진단

### 뒤 10 줄 전체 평균의 위험



실제로 관련된 1~3 개 line 만 강하고  
나머지가 무관하면 Mean 에서 신호가 희석될 수 있음

#### 가장 현실적인 예상

전체 Layer/Head 평균  
 $AUC \approx 0.55 \sim 0.65$

일부 Middle/Late Layer  
 $AUC \approx 0.60 \sim 0.70$

#### 특정 Head

$AUC > 0.70$  인 Head 가 일부 존재 가능

단, 사후 선택은 “탐색 분석” 으로만 해석

#### Secondary score

Top-3 mean  
Max  
Target-token normalized

→ 신호 희석 / 길이 효과 진단

#### 해석 원칙

Mean 이 약해도 Top-3/Max 가 강하면  
“라인 어텐션 실패” 가 아니라  
“전체 평균 aggregation 이 희석” 했을 가능성

## 7. 현재 코드 구조

Parser → Window → 입력 /Span 구성 → Attention 추출 → Line score/AUC

1

### Parser / 전처리

현재 코드

- 원본 Thunderbird 로그를 CSV row 로 변환
- line\_id, label, timestamp, user/location/component/content 저장

2

### Window Builder

현재 코드

- 파싱 CSV 를 읽어 21 줄 Window(JSONL) 생성
- 실험에서는 Target 을 Line 10 에 고정하고 Alert/Normal 후보 구성

3

### Input / Span Builder

현재 코드

- 5 개 field 를 [LINE id] 형식의 모델 입력 text 로 연결
- 각 field 의 start/end span 저장 → offset\_mapping 으로 token→line 연결
- target/prefix/suffix/total token 통계도 여기서 계산

4

### Attention Extractor

실험 연결

- Qwen forward 에서 output\_attentions=True
- Layer 별 Attention tensor [B, H, T, T] 확보
- Head 별 token attention 을 line 단위  $R_{ij}$  로 집계

5

### Score / AUC Evaluator

실험 연결

- $R_{11,10} \dots R_{20,10} \rightarrow S_{10}(\text{mean})$  계산
- Head 평균 → Layer 평균 → Window 별 최종  $S_{10}$  생성
- Alert  $S_{10}$  vs Normal  $S_{10}$  로 AUC 계산

최종 데이터 흐름

raw log → parsed.csv → windows.jsonl → token/line map →  $S_{10} \rightarrow \text{AUC}$

핵심 : 현재 작성한 Parser / Window / Pipeline 코드를 그대로 살리고 , 그 뒤에 Attention 추출과 AUC 평가 모듈을 연결하는 구조

---

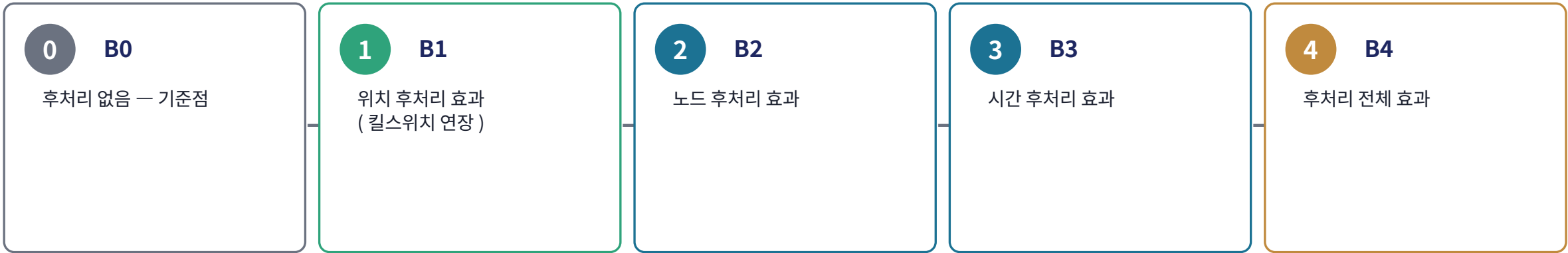
PART D

# 평가 설계

압축기가 원인을 잘 보존하는지 어떻게 판정하나 — 신호별 기여 (Ablation) 와 압축 로그로 원인을 맞히는지 (K 지선다 RCA) 를 재는 평가 파이프라인 .

# 어느 신호가 원인 보존에 기여하나

압축기 = [ 어텐션으로 줄 거르기 ] → [ 위치 · 노드 · 시간 후처리 ]. 후처리를 하나씩 켜며 보존율이 오르는지 본다.



## 핵심 대비 · B0 vs B1

어텐션만 vs 어텐션 + 위치 . 유의하게 오르면 → ‘위치가 원인 보존에 실제 기여’ = 킬스위치의 보존 단계 연장 증거 .

## 채점 · 판정

각 조합 보존율 → 같은 원인줄을 두 방식이 처리 ( 짝 ) → McNemar + BH 보정 . 기존 KPI 코드 재할용 .

킬스위치 AUC = 신호의 ‘구별력’ ( 압축 전 , 원본 ). **ablation** = 신호의 ‘보존 기여’ ( 압축 후 ). 재료 검증 vs 완제품 검증 — 둘 다 필요 .

B0 = C 의 1 차 실험 ( 어텐션만 ) 과 같은 지점 → 두 실험이 이어짐 .

## 압축된 로그로 원인을 맞히나 — 코드 흐름 (1/2)

### 1 로그 로딩 · 파싱 load\_log()

Nezha 로그 CSV → PodName( 서비스 ) · Log( 메시지 ) 파싱 . 25,574 줄 로드 확인 .

### 2 정답 줄 찾기 find\_cause\_lines()

카탈로그 라벨 ( 서비스 + 이벤트 ) 로 원인 줄 검색 . → 정답 2,418 줄 발견 .

### 3 오답 보기 뽑기 make\_distractors()

같은 계열 다른 서비스의 대표 이벤트 K-1 개 . 단어매칭 방지 .

### 4 문제 조립 build\_question()

정답 1+ 오답 K-1 섞기 → { 보기 , 정답번호 } 딕셔너리 .

```
# 2. 정답 줄 찾기
def find_cause_lines(df, svc, pat):
    hit = df[
        (df['service'] == svc) &
        (df['msg'].str.contains(pat))
    ]
    return hit.index.tolist()
```

→ 라벨을 로그의 실제 줄 인덱스로 변환

### 생성된 K 지선다 ( 실제 출력 )

Q. 이 로그의 장애 원인 이벤트는 ?

- ① **productcatalog: Query product...** ✓
- ② frontend: Request complete
- ③ currency: Handles carrying
- ④ payment: Transaction ...

## 프롬프트 → 채점 → 압축 연결 (2/2)

### 5 프롬프트 build\_prompt()

로그 + 보기를 LLM 입력 형식으로, ‘번호 하나만 답하라’ 강제.

### 6 채점 grade()

LLM 답에서 숫자 추출 → 정답번호와 대조. 정답 = 라벨이라 자동 채점.

### 7 압축 자리 compress()

지금은 원본 (none). C 압축기 오면 이 함수만 교체 → 압축 로그로 풀기.

```
# 7. C 압축기 들어올 자리
def compress(df, method='none',
             keep_lines=None):
    if method=='none':
        return df # 지금 : 원본
    return df.loc[keep_lines]
# ↑ 압축기가 준 살릴 줄만
```

→ 나머지 (문제 · 채점) 는 그대로. 압축기만 끼우면 원본 vs 압축 비교 완성.

### 최종 판정

같은 문제를 원본 로그 vs 압축 로그가 각각 풀기 = 짝 → **McNemar + BH.** ‘압축해도 원인을 맞히나 (delta)’ 를 잴다.